

Architecting a Secure Gentoo Desktop: An Advanced Installation with SELinux, ZFS, and a Custom CachyOS Kernel

Pre-Installation: Live Environment Preparation and Hardware Sanitization

The foundation of any successful Gentoo installation, particularly one involving advanced technologies like ZFS and ZFSBootMenu, is a meticulously prepared live environment. This initial phase is not merely about booting an operating system; it is about establishing a stable, tool-rich, and correctly configured workspace from which the new Gentoo system will be built. Rushing or skipping these preparatory steps invariably leads to subtle but critical errors that can manifest as unbootable systems or corrupted storage pools. This section details the essential procedures for preparing the live environment, sanitizing the target hardware, generating a critical ZFS identifier, and ensuring network connectivity for the subsequent bootstrap process. Each step is designed to eliminate common pitfalls and set the stage for a smooth and predictable installation.

The first decision is the selection of the live environment itself. While many distributions offer live images capable of handling this task, such as Arch-based SystemRescue or Parted Magic [1](#) [6](#), the official Gentoo Admin CD is highly recommended. It is purpose-built for Gentoo installations, comes pre-loaded with many of the required tools, and guarantees compatibility with Gentoo's infrastructure and package manager [18](#). Regardless of the chosen live medium, the primary objective is to ensure that the necessary userspace utilities are available. For managing ZFS, the `zfs` toolset is indispensable. For configuring the UEFI bootloader, `efibootmgr` is required. Finally, utilities like `sg3_utils` for low-level disk commands, and `curl` or `wget` for downloading files like ZFSBootMenu, are highly beneficial [2](#) [5](#). If your live environment uses `pacman` (like Arch-based ones), you would install these with `pacman -S zfs efibootmgr sg3_utils curl wget`. If it uses Gentoo's own `emerge`, the command would be `emerge sys-fs/zfs sys-boot/efibootmgr sys-fs/sg3_utils net-misc/curl net-misc/wget`.

With the live environment booted and tools installed, the next critical task is the sanitization of the target physical hardware. For this installation, the goal is to create a striped RAID0 ZFS pool from two NVMe SSDs. To prevent ZFS from encountering conflicting metadata or partition tables from a previous installation or OS, these drives must be completely wiped clean before pool creation [9](#). The `sgdisk` utility is the standard tool for this job. By running `sgdisk --zap-all /dev/nvme0n1` followed by `sgdisk --zap-all /dev/nvme1n1`, you destroy all partitions, protective MBRs, and unique identifiers on the specified devices [17](#). After executing these commands, it is imperative to verify that the drives are indeed clean. Using `lsblk /dev/nvme0n1 /dev/nvme1n1` should show no partition columns, indicating that ZFS will have a pristine slate upon which to build its own metadata [8](#).

A subtler but absolutely non-negotiable prerequisite for any ZFS-on-root setup is the generation and persistence of a stable `hostid`. The `hostid` is a unique 32-bit number that ZFS embeds into the metadata of every pool it creates [17](#). During the boot process, the system consults this ID to identify its host. If the `hostid` changes between boots—for instance, if it is not persisted across reboots or during a fresh installation—ZFS may fail to automatically import the root pool because it sees the pool as belonging to a different machine [8](#). This can lead to a frustrating boot failure where the system cannot find its root filesystem. To prevent this, the `zgenhostid` tool must be used in the live environment *before* any ZFS operations are performed. A common practice is to generate a fixed, arbitrary hexadecimal ID. This is accomplished with the command `zgenhostid -f 0x00bab10c`. The value `0x00bab10c` is arbitrary; any unique 8-digit hexadecimal number is acceptable. Once generated, this ID must be written to the `/etc/hostid` file within the new, future Gentoo filesystem. Since we are still operating from the live environment, this file needs to be created at the location relative to our mount point, which is `/mnt`. Therefore, the command `echo -ne '\x00\xba\xbl\x0c' > /mnt/etc/hostid` writes the binary representation of the ID to the correct path [19](#). Verifying the write was successful with `cat /etc/hostid` ensures the operation was completed correctly. This single action guarantees that the ZFS pool's metadata will always refer to the same host, preventing catastrophic import-related boot failures [17](#).

Finally, before proceeding to chroot into the newly extracted Gentoo environment, network connectivity must be established. Portage, Gentoo's package management framework, requires internet access to synchronize its package database (`emerge-webrsync` or `emerge --sync`). This synchronization process involves contacting Gentoo mirrors via DNS, which relies on the resolver configuration. The simplest and most reliable way to provide this configuration is to copy the nameserver information from the live environment's `resolv.conf` file into the target system's configuration file. This

is achieved with the command `cp /etc/resolv.conf /mnt/etc/`. Without this step, any attempt to sync Portage inside the chroot will fail with "temporary failure in name resolution" errors, effectively halting the entire installation process [5](#) . At this point, the live environment is fully prepared. The stage3 tarball can be downloaded and extracted, the chroot environment can be mounted, and the Gentoo installation can proceed without interruption. The careful execution of these preparatory steps lays the groundwork for a smooth and successful installation of a complex, custom-built Gentoo system.

Step	Command(s)	Description
Select Live Environment	Gentoo Admin CD, SystemRescue, or Arch-based image with ZFS support 1	Choose an environment with the necessary package manager (emerge or pacman) and tools.
Install Live Tools	<code>pacman -S zfs efibootmgr sg3_utils</code> <code>curl wget</code> 5	Installs essential utilities for ZFS management, UEFI boot entry modification, and file downloads.
Wipe Target Disks	<code>sgdisk --zap-all /dev/nvme0n1</code> <code>sgdisk --zap-all /dev/nvme1n1</code> 9	Destroys all partition tables and protective MBRs on the target NVMe drives to ensure they are clean for ZFS.
Generate HostID	<code>zgenhostid -f 0x00bab10c</code> 17	Generates a fixed, arbitrary 32-bit host identifier for ZFS to ensure consistent pool import.
Persist HostID	<code>echo -ne '\x00\xba\xbl\x0c' > /mnt/etc/hostid</code> 19	Writes the generated host ID to the /etc/hostid file in the target filesystem.
Enable Networking	<code>cp /etc/resolv.conf /mnt/etc/</code> 5	Copies the live environment's DNS resolver configuration to the target system, enabling <code>emerge --sync</code> inside the chroot.

Base System Bootstrap and Portage Configuration

With the live environment meticulously prepared, the core of the Gentoo installation begins: the bootstrap process. This phase involves extracting the minimal base operating system from a stage3 tarball, mounting essential virtual filesystems to create a functional chroot environment, and performing the critical initial configurations of Portage—the heart of Gentoo's package management system. The precision of this setup directly influences the compilation of the kernel, ZFS modules, and all subsequent software. This section provides a comprehensive walkthrough of the bootstrap process, detailing the extraction of the stage3 archive, the meticulous configuration of Portage and `make.conf`, and the finalization of the chroot environment before proceeding to the most complex part of the installation: kernel compilation.

The first step in the bootstrap is to obtain and extract the stage3 tarball. A stage3 archive is a pre-compiled snapshot of a minimal Gentoo system, containing the essential

directories and files needed to begin building the rest of the OS [6](#) . Given the user's specific requirement for a hardened SELinux system, it is crucial to download the corresponding stage3 tarball, such as `stage3-amd64-hardened-selinux-systemd-YYYYMMDDT000000Z.tar.xz` [10](#) . The extraction command must be executed from the parent directory of your target mount point (assumed to be `/mnt`). The command is `tar xpf /path/to/stage3-amd64-hardened-selinux-systemd-..tar.xz -C /mnt --xattrs-include='</em>.<em>' --numeric-owner`. The flags `--xattrs-include='</em>.*'` and `--numeric-owner` are vital; they preserve extended attributes and ensure correct numeric ownership of files and directories, which is essential for features like SELinux contexts and ZFS functionality [20](#) . After a successful extraction, the directory structure under `/mnt` should now resemble a standard Gentoo root filesystem, including directories like `/bin`, `/etc`, `/usr`, and `/var`.

Next, several special-purpose virtual filesystems must be mounted into the newly extracted environment. These filesystems provide the kernel's interface to the hardware and running processes and are required for the chroot environment to function correctly [12](#) . They are mounted using the `mount --bind` command. The following mounts are necessary:

- ``mount --bind /proc /mnt/proc``
- ``mount --bind /sys /mnt/sys``
- ``mount --bind /dev /mnt/dev``
- ``mount --bind /run /mnt/run``

These commands make the device nodes, process information, and system runtime sockets from the live environment accessible from within the chroot, allowing processes inside to interact with the hardware and each other [5](#) . Special care must be taken with `/dev/shm`. If it is a symbolic link in the live environment, it should be removed and replaced with a proper tmpfs mount to avoid issues with certain builds. The commands to handle this are:

```
if ; then
    rm /mnt/dev/shm
fi
mkdir -p /mnt/dev/shm
mount -t tmpfs -o nosuid,nodev,noexec shm /mnt/dev/shm
chmod 1777 /mnt/dev/shm
```

This ensures a secure and functional temporary filesystem for the chroot session.

Once the filesystems are bound, the chroot environment is entered using the command `chroot /mnt /bin/bash`. This command changes the root directory (/) for the current shell session to /mnt, effectively placing you inside the new, uninitialized Gentoo system [6](#). Upon entering the chroot, it is good practice to initialize the environment by sourcing the profile script with `source /etc/profile`. This ensures that environment variables like `PATH` are set up according to Gentoo standards. Setting a custom prompt, such as `export PS1="(chroot) ${PS1}"`, helps to visually distinguish the chroot session from the live environment [6](#).

Inside the chroot, the first order of business is to synchronize the Portage tree. Portage is Gentoo's package management framework, and its tree contains metadata about all available packages, their dependencies, and build configurations. Synchronizing it ensures you have the latest information. This is done with the command `emerge-webrsync`. This command downloads a compressed snapshot of the Portage tree from a mirror, which is significantly faster than a full `rsync` for the initial setup [5](#). Alternatively, if network conditions are favorable, `emerge --sync` can be used, which performs a more thorough update via `rsync`. Following the sync, the next step is to select a system profile. Profiles define the default settings for your Gentoo installation, including the default USE flags, accepted keywords, and `package.mask` settings. You can view the list of available profiles with `eselect profile list`. The output will show numbered entries. You should select a profile appropriate for your intended system type (e.g., desktop) and architecture. For example, `eselect profile set 17` might select a generic desktop profile for AMD64.

The most critical configuration file created during this phase is `/etc/portage/make.conf`. This file controls how packages are compiled and installed. For a ZFS-on-root system with `systemd` and native encryption, a properly configured `make.conf` is essential. Based on the user's specifications and best practices, the configuration should include the following directives:

```
## File: /etc/portage/make.conf
CHOST="x86_64-pc-linux-gnu"
CFLAGS="-march=native -O2 -pipe"
CXXFLAGS="${CFLAGS}"
MAKEOPTS="-j$(nproc)"

USE="systemd selinux hardened rootfs"

ACCEPT_LICENSE="*"
```

Let's break down the key directives in this configuration:

- ``CHOST``: Specifies the target architecture for binaries. Gentoo typically auto-detects this correctly.
- ``CFLAGS`` and ``CXXFLAGS``: Define the compiler flags. ``-march=native`` optimizes the compiled code for the specific CPU model, while ``-O2`` enables a balanced level of optimization and ``-pipe`` uses pipes instead of temporary files for communication between compiler stages, improving compile speed [22](#).
- ``MAKEOPTS``: Sets the ``-j`` flag for ``make``. Using ``-j$(nproc)`` tells ``emerge`` to use as many parallel jobs as there are CPU threads, which dramatically speeds up the compilation of large packages like the kernel and ZFS [21](#).
- ``USE="systemd selinux hardened rootfs"``: This line is absolutely critical for this installation. The ``systemd`` USE flag enables all the necessary dependencies for building and running a systemd-based system. The ``selinux`` and ``hardened`` USE flags enable the security features from the selected stage3. The ``rootfs`` USE flag is mandatory for the ``sys-fs/zfs`` package to enable features required for booting from a ZFS root dataset [21](#). Without it, the system will fail to boot.
- ``ACCEPT_LICENSE="*"``: Instructs Portage to accept all licenses for packages, which simplifies the installation process by avoiding license acceptance prompts.

Finally, the base system packages must be installed. This includes the core Portage tools themselves, essential system utilities, `shadow` for password management, and `util-linux`. Since the user has specified `systemd` as the sole init system, the installation command would be: `emerge -av sys-apps/portage sys-apps/systemd sys-apps/util-linux sys-auth/shadow net-misc/networkmanager sys-process/cronie`. This command installs the minimum set of packages required for a functional, albeit empty, base system. At this point, the bootstrap process is complete. The chroot environment is fully configured with a synchronized Portage tree, the correct system profile, a properly tuned `make.conf`, and a base set of system utilities. The stage is now set for the most complex part of the installation: compiling the kernel and ZFS modules.

Kernel Compilation and ZFS Module Integration Pathways

The selection and compilation of the Linux kernel and its associated ZFS modules represent the most technically demanding phase of the Gentoo installation. Gentoo's

philosophy of providing maximum control and customization means the user is responsible for this process. For a ZFS-on-root system, the kernel must either have native ZFS support enabled or be paired with the ZFS kernel modules built via DKMS (Dynamic Kernel Modules Infrastructure). This section outlines two distinct pathways for achieving this, reflecting the user's request to explore both the traditional DKMS approach and the experimental kernel-builtin ZFS method. Both paths require careful configuration of the `make.conf` file and the use of Gentoo's package manager, `emerge`. The choice between them depends on the stability and feature completeness of the chosen kernel source, specifically `sys-kernel/cachyos-sources` with the `kernel-builtin-zfs` USE flag.

Pathway A: Traditional DKMS-Based ZFS (Recommended and Stable)

The DKMS-based approach is the officially supported and well-tested method for integrating ZFS with a Gentoo system. It works by compiling the ZFS kernel modules against the currently running kernel's headers and storing them in a central repository. When the kernel is updated, DKMS automatically recompiles the modules for the new kernel, ensuring compatibility. This pathway is highly reliable and should be used unless the kernel-builtin option proves successful.

First, you must install the necessary packages for kernel compilation and module building. Inside the chroot environment, run: `emerge -av sys-kernel/dkms sys-kernel/linux-headers sys-kernel/gentoo-kernel sys-kernel/linux-firmware`. This command installs `dkms` itself, the `linux-headers` package which provides the C header files needed to compile external kernel modules, a binary kernel package (`gentoo-kernel`) for simplicity, and `linux-firmware` which contains firmware blobs for various hardware components, including NVMe drives [22](#). Using a pre-built binary kernel like `gentoo-kernel` is often the fastest way to get a working system, as it avoids the lengthy process of manually compiling a kernel from source.

With the build environment ready, the next step is to emerge the ZFS userspace tools. Run: `emerge -av sys-fs/zfs`. During the compilation process, Portage will trigger DKMS to build the ZFS, SPL (Solaris Porting Layer), and ZSTD compression modules against the active kernel headers [16](#). This compilation can be quite resource-intensive and time-consuming, especially on less powerful CPUs, potentially taking anywhere from 10 to 30 minutes. It is important to ensure that `/var/tmp` has sufficient free space (at least 4GB is recommended) to hold the intermediate build files [14](#). You can monitor the progress and check for errors by examining the log files located in `/var/log/dkms/`. If the `emerge` process completes successfully, the ZFS modules are now compiled and registered with DKMS, ready to be loaded by the kernel at boot time. The

USE="rootfs" flag, configured in the `make.conf` during the bootstrap phase, is what enables the specific features needed for a ZFS root filesystem [21](#).

Pathway B: Experimental Kernel-Builtin ZFS

The second pathway aims to integrate ZFS directly into the mainline Linux kernel, eliminating the need for out-of-tree modules and the associated DKMS infrastructure. This can lead to better performance and simpler dependency management. However, this feature is relatively new and its stability and completeness depend heavily on the specific kernel version and distribution patches.

The user has specified the use of `sys-kernel/cachyos-sources`. CachyOS is an Arch-based distribution that provides highly optimized and cutting-edge kernel sources, making it a prime candidate for testing new features like built-in ZFS [2](#) [3](#). The 6.18 long-term kernel series is also a modern choice, balancing new features with stability [24](#). The success of this path hinges on whether the `kernel-builtin-zfs` USE flag, when applied to `sys-fs/zfs`, correctly compiles the package against the kernel source expecting in-kernel ZFS support.

To attempt this pathway, you must first install the kernel sources and firmware: `emerge -av "=sys-kernel/cachyos-sources-6.18*" sys-kernel/linux-firmware`. You will then need to add the USE flag to your `/etc/portage/package.use` file. Create this file if it doesn't exist and add the following line: `sys-fs/zfs kernel-builtin-zfs`. This directive tells Portage to build ZFS with the expectation that the kernel has native support.

Now, emerge ZFS with this new configuration: `emerge -av sys-fs/zfs`. The compilation process will differ from the DKMS path. Instead of invoking DKMS, it will attempt to link against the kernel source tree. If the `cachyos-sources` kernel for the 6.18 series has the necessary headers and symbols exposed, the compilation will succeed, and you will have a ZFS userspace implementation that communicates with the kernel via the in-kernel ZFS module. If, however, the kernel does not have the required support or the USE flag is not fully implemented, the emerge process will fail with compilation errors. In this case, you should remove the `kernel-builtin-zfs` flag from your `package.use` file and revert to the stable DKMS-based pathway outlined in Pathway A.

Regardless of the chosen pathway, after successfully emerging `sys-fs/zfs`, the final step is to configure the init system services. Since the user has specified `systemd` as the sole init system, you must enable the necessary ZFS-related services. These services

handle the automatic import of pools, mounting of datasets, and monitoring of pool health. The commands to enable them are:

- ``systemctl enable zfs-import-cache.service``: This service imports any ZFS pools listed in ``/etc/zfs/zpool.cache`` early in the boot process, which is essential for finding the root pool.
- ``systemctl enable zfs-mount.service``: This service mounts all ZFS datasets whose ``mountpoint`` property is not set to ``none`` or ``noauto``.
- ``systemctl enable zfs-zed.service``: This starts the ZFS Event Daemon, which can be configured to send notifications about pool events, such as disk failures or hot spare replacements [22](#).

Feature	Pathway A: DKMS-Based	Pathway B: Kernel-Builtin ZFS
Kernel Source	<code>sys-kernel/gentoo-kernel</code> or <code>sys-kernel/gentoo-sources</code> 22	<code>sys-kernel/cachyos-sources</code> (user-specified) 3
USE Flag	<code>USE="systemd selinux hardened rootfs"</code> (for ZFS) 21	<code>USE="systemd selinux hardened rootfs kernel-builtin-zfs"</code> (for ZFS) 22
Module Build Tool	<code>sys-kernel/dkms</code> 16	Direct linking against kernel source (no DKMS)
Reliability	High. Officially supported, well-tested method. 22	Experimental. Depends on kernel source implementation. May fail to compile. 3
Maintenance	Automatic. DKMS recompiles modules on kernel updates. 22	Manual. Requires rebuilding ZFS if kernel changes or if the feature becomes unstable. 3
Dependencies	<code>sys-kernel/dkms</code> , <code>sys-kernel/linux-headers</code> 22	Only <code>sys-kernel/linux-headers</code>
Recommendation	Default choice for a production system. 22	Advanced use only. Recommended for experimentation. 3

By carefully selecting and configuring the kernel and ZFS modules, you are laying the lowest-level foundation for your entire operating system. The decision between the stable DKMS path and the potentially streamlined but unproven built-in ZFS path is a key architectural choice that defines the system's long-term maintainability and complexity.

Storage Initialization: Creating the Encrypted ZFS Pool and Datasets

With the base Gentoo system and kernel/ZFS modules in place, the next major step is to construct the primary storage substrate: the ZFS pool. This phase involves aggregating

the two physical NVMe SSDs into a single logical volume, applying full-disk encryption directly within ZFS, and creating a hierarchical structure of datasets to organize the operating system and user data. This process fundamentally differs from traditional Linux installations that rely on partitioning tools like `fdisk` and filesystem creators like `mkfs.ext4`. Here, ZFS acts as both the volume manager and the filesystem, offering advanced features like integrated encryption, data integrity, and efficient snapshots from the outset. This section provides a detailed walkthrough of creating the striped RAID0 pool, enabling native encryption, and defining the dataset hierarchy that will serve as the home for the Gentoo installation.

The first command in this phase is `zpool create`. This command aggregates physical devices into a single virtual storage pool. The user's requirement for a RAID0 striped pool across two NVMe drives translates directly to using the `raid0` vdev type. Assuming the two NVMe drives are identified as `/dev/nvme0n1` and `/dev/nvme1n1`, the command to create the pool named `rpool` would be: `zpool create -f rpool raid0 /dev/nvme0n1 /dev/nvme1n1`. The `-f` flag forces the creation, ignoring warnings about the lack of redundancy in a RAID0 configuration. This command instructs ZFS to stripe data evenly across both drives, maximizing read and write performance at the expense of fault tolerance; the failure of either drive will result in the complete loss of the pool [22](#). It is crucial to verify the device names (`/dev/nvme0n1`, `/dev/nvme1n1`) using a tool like `lsblk` before executing this command, as specifying the wrong devices will result in irreversible data loss.

Immediately after creating the pool, the next critical step is to enable encryption. The user's explicit requirement is to use native ZFS encryption, bypassing any outer-layer encryption like LUKS. ZFS encryption is enabled on a per-dataset basis. To create an encrypted root dataset, you first need to create a child dataset of the newly formed `rpool`. The command `zfs create -o encryption=on -o keyformat=passphrase rpool/R00T/default` accomplishes this. Let's dissect this command:

- ``zfs create``: Creates a new filesystem (dataset).
- ``-o encryption=on``: This property enables encryption for the new dataset.
- ``-o keyformat=passphrase``: Specifies that the key will be derived from a passphrase provided by the user. An alternative is ``keyformat=raw`` for a raw keyfile, which is useful for automation but carries higher security risks [22](#).

When this command is executed, ZFS will prompt for a passphrase. This passphrase will be used to derive the actual AES-256-GCM key that encrypts the dataset's contents. It is

imperative to choose a strong, memorable passphrase and store it securely, as losing it will result in permanent data loss. ZFS will then ask for a confirmation of the passphrase.

At this point, the encrypted root pool is created, but it is not yet mounted or usable. The `rpool/ROOT/default` dataset is the top-level dataset that will contain the entire operating system. Before moving on, it is wise to review the state of the pool. Running `zpool status` will display information about the pool's health, vdev layout, and any potential issues. Running `zfs list -r rpool` will show a recursive listing of all datasets within `rpool`, confirming that `rpool/ROOT/default` exists and has the expected properties, including the `encryption` property being set to `AES-256-GCM`.

With the root dataset created, it is beneficial to establish a clear hierarchy of datasets for the rest of the system. ZFS excels at this, allowing you to create datasets for `/home`, `/var`, and other standard directories. Each dataset can have its own unique properties, such as different mount points, quotas, or even separate encryption keys. For this setup, a logical structure would be to create datasets as children of the root dataset. For example:

- ``zfs create rpool/ROOT/default/home``
- ``zfs create rpool/ROOT/default/var``
- ``zfs create rpool/ROOT/default/var/log``
- ``zfs create rpool/ROOT/default/usr/local``

Creating these datasets establishes a clean separation of concerns. The `/home` dataset can be snapshotted and cloned independently of the system root, `/var/log` can be managed separately for log rotation policies, and so on. For datasets that should not be automatically mounted by the ZFS mount service (like the root dataset itself, which will be handled specially by the boot process), you can set the `canmount` property to `noauto`. For instance, `zfs set canmount=noauto rpool/ROOT/default`. This is a best practice for boot environments, as it prevents ZFS from trying to mount them twice [22](#). Other datasets like `rpool/ROOT/default/home` will have their `mountpoint` property automatically set to `/home` when inherited from the parent, and ZFS will mount them correctly at boot time.

One of the most important properties to set on the root dataset is `bootfs`. This property tells the system which dataset within the pool contains a valid operating system and should be booted from. To designate our newly created dataset as the system's bootable root, run: `zpool set bootfs=rpool/ROOT/default rpool`. This simple command embeds the bootable path into the pool's configuration, making it discoverable by the bootloader.

Finally, the swap space must be configured. On a ZFS system, swap is typically allocated as a dedicated Zvol (ZFS volume). A Zvol presents a block device interface to the kernel, making it ideal for swap. First, create the Zvol with a desired size, for example, 16GB: `zfs create -V 16G -o compression=zstd -o logbias=throughput -o sync=always rpool/swap`. Let's examine the properties set here:

- `-V 16G``: Creates a 16GB block device.
- `-o compression=zstd``: Enables compression on the Zvol, saving space.
- `-o logbias=throughput``: Instructs ZFS to optimize the Zvol's write placement for throughput rather than latency, which is generally preferred for swap activity [22](#).
- `-o sync=always``: Ensures that all writes to the swap Zvol are forced to stable storage immediately, preventing data corruption in the event of a crash. This is a safe but slightly slower setting compared to `sync=standard`` or `sync=disabled`` [22](#).

After creating the Zvol, format it as a swap area with `mkswap /dev/zvol/rpool/swap` and activate it with `swapon /dev/zvol/rpool/swap`. This Zvol will serve as the system's swap space.

Command	Description	Key Properties Used
<code>zpool create -f rpool raid0 /dev/nvme0n1 /dev/nvme1n1</code>	Creates a striped RAID0 pool named 'rpool' from two NVMe devices. 22	raid0
<code>zfs create -o encryption=on -o keyformat=passphrase rpool/R00T/default</code>	Creates an encrypted root dataset, prompting for a passphrase. 22	encryption=on, keyformat=passphrase
<code>zpool set bootfs=rpool/R00T/default rpool</code>	Designates the specified dataset as the system's bootable root. 22	bootfs
<code>zfs create rpool/R00T/default/home</code>	Creates a child dataset for user home directories. 22	Inherits properties from parent.
<code>zfs create -V 16G -o logbias=throughput -o sync=always rpool/swap</code>	Creates a 16GB ZFS volume for swap with optimized properties. 22	-V (size), logbias=throughput, sync=always

Once these commands are executed, the storage subsystem is fully initialized. The encrypted RAID0 pool is created, the root dataset is designated, a logical hierarchy of datasets is in place, and a dedicated Zvol for swap has been provisioned. The system is now ready for the final piece of the puzzle: configuring the bootloader to manage this complex ZFS-based setup.

Bootloader Configuration: Integrating ZFSBootMenu and UEFI Registration

The final and arguably most crucial component of this specialized Gentoo installation is the bootloader. Given the use of a ZFS root filesystem with native encryption and the goal of managing bootable snapshots, the traditional GRUB bootloader is insufficient. The role of orchestrating the boot process falls to ZFSBootMenu (ZBM), a specialized bootloader designed specifically for ZFS-based systems. ZBM's primary function is to intervene very early in the boot sequence, before the host kernel is even loaded. It uses its own embedded kernel and initramfs to scan for ZFS pools, present a menu to unlock encrypted pools, allow the selection of different bootable snapshots, and finally hand off control to the selected operating system's init process [8](#) [17](#). This section details the process of downloading, installing, and configuring ZFSBootMenu to bring together all the architectural elements of this guide into a cohesive and manageable whole.

The first step is to obtain the ZFSBootMenu binaries. These are pre-compiled EFI applications that are not distributed through the Gentoo Portage tree. The official ZBM releases page on GitHub hosts the latest versions. You should download the `.zip` archive corresponding to the latest release. Inside the zip file, you will find several subdirectories, including `bin`, which contains the main ZBM EFI application (`zfsbootmenu.efi`), and `keys`, which contains cryptographic keys for signing ZBM's internal components. For simplicity, especially in a single-user, non-Secure-Boot environment, the focus will be on using the pre-signed binaries included in the release.

With the ZIP archive downloaded, you need to extract its contents and copy them to the appropriate locations on your newly created filesystem. The EFI System Partition (ESP), which is typically mounted at `/boot/efi`, is where all UEFI bootloader applications reside. First, ensure the ESP is mounted. Then, create a dedicated directory for ZBM within the ESP, for example, `/boot/efi/loader/entries/`. The exact path may vary depending on your UEFI firmware's expectations, but a common convention is to place the main executable in `/boot/efi/` and configuration in `/boot/efi/loader/`. Extract the `zfsbootmenu.efi` binary from the ZIP and copy it to the ESP's root directory, e.g., `cp zfsbootmenu-*/bin/x86_64-efi/zfsbootmenu.efi /boot/efi/`. You will also need to copy the `zfsbootmenu.conf` file, which will be created next, to a location where ZBM can find it, typically `/etc/zfsbootmenu/zfsbootmenu.conf`.

The heart of ZBM's functionality lies in its configuration file. This file, located at `/etc/zfsbootmenu/zfsbootmenu.conf` inside the chroot, dictates its behavior. A minimal but functional configuration for this setup would be:

```
## File: /etc/zfsbootmenu/zfsbootmenu.conf
[Global]
# Disable automatic image generation. We are using a static boot menu.
ManageImages = false
# Prefix for generated snapshot names. Not strictly necessary for a simple
GenerationPrefix = zbm-

[Kernel]
# Specify the kernel command line arguments to pass to the booted kernel.
# `root=ZFS=rpool/ROOT/default` tells the kernel to mount our ZFS dataset
# `ro` mounts the root filesystem as read-only initially.
# `quiet` reduces boot-time console messages.
# `zbm.show=auto` forces the ZBM menu to appear automatically.
CommandLine = "root=ZFS=rpool/ROOT/default ro quiet zbm.show=auto"
```

Let's analyze the key sections and parameters in this configuration:

- `[Global]`: This section contains global settings for ZBM. `ManageImages = false` is a critical setting. It tells ZBM not to automatically generate new kernel/initramfs images for every boot, which is unnecessary since we are relying on the host system's kernel. ZBM's job is purely to unlock and boot the existing one.
- `[Kernel]`: This section defines the kernel command line passed to the booted kernel. The `CommandLine` is the most important parameter. `root=ZFS=rpool/ROOT/default` is the ZFS-specific syntax that tells the kernel to locate the `default` dataset within the `rpool` and use it as the root filesystem. The `zbm.show=auto` parameter ensures that the boot menu is displayed automatically, which is essential for snapshot selection. Without it, the system might boot the default snapshot immediately, defeating the purpose of having a menu.

After creating the configuration file inside the chroot, you must exit the chroot environment (`exit`) and remount your `/boot/efi` partition to make the files visible outside of it. The final step is to register ZBM as a boot entry with the UEFI firmware. This is done using the `efibootmgr` tool. You need to create a new boot entry that points to the `zfsbootmenu.efi` file you copied to the ESP. The exact vendor GUID and filename may vary, but a typical command would be: `efibootmgr -c -L "Gentoo ZFS" -l '\zfsbootmenu.efi' -p \EFI\zfsbootmenu.efi`. This command

creates a new boot entry (-c), assigns it a label (-L "Gentoo ZFS"), and specifies the path to the EFI application (-l). After running this command, you can verify the new entry with `efibootmgr -v`. You may want to reorder the boot entries to ensure that "Gentoo ZFS" is the first boot option.

Once the system is rebooted, ZBM will intercept the boot process. If the ZFS pool is encrypted, ZBM will pause and prompt you for the passphrase you set earlier. After unlocking the pool, it will scan for bootable datasets (those with the `bootfs` property set) and present them in a menu. In our current setup, it will likely show a single entry pointing to `rpool/R00T/default`. Selecting this entry will cause ZBM to mount the dataset and execute the host system's `/sbin/init` (`systemd`), completing the boot process.

This entire mechanism provides powerful recovery capabilities. If a new software installation renders the system unbootable, you can simply create a new snapshot of a working system state (`zfs snapshot rpool/R00T/snapshot-backup`) and boot into it. On the next reboot, ZBM will detect the new snapshot and present it as an option in its menu. By selecting the working snapshot, you can easily recover your system. If ZBM fails to start or you need to perform deeper repairs, holding down a modifier key (such as `Shift`) during ZBM startup can drop you into a ZFS shell. From this emergency shell, you can run diagnostic commands like `zpool status` to check for problems, `zfs list` to see available snapshots, or `zpool import` to manually import the pool, giving you full administrative control over the storage layer [17](#).

Final System Setup, SELinux Policy Tuning, and Reboot Verification

The preceding sections have laid the architectural foundation for a sophisticated Gentoo Linux system. The storage pool is created, the kernel and ZFS modules are compiled, the bootloader is configured, and the base operating system is in place. The final phase of the installation involves transitioning from the installation media to the newly created system, performing essential post-boot configuration, and verifying that all components are functioning as intended. This stage includes setting the root password, creating a standard user account, installing a display server, and ensuring the system can boot correctly from its ZFS root. It is a critical period for troubleshooting and validation, ensuring the complex interplay between ZFS, ZFSBootMenu, and `systemd` is seamless.

The first action upon a successful boot into the new Gentoo system is to gain administrative privileges. Since you booted from ZFSBootMenu, you will be presented with a login prompt. Log in as the `root` user. The first task is to set a secure password for the root account if one hasn't already been set during the installation process. This is done with the `passwd` command. Next, it is a security best practice to create a standard user account for daily operations and disable direct root logins. You can create a new user with `useradd -m -G wheel,audio,video -s /bin/bash username`, replacing `username` with the desired login name. The `-m` flag creates a home directory, `-G` adds the user to supplementary groups for hardware access (like audio and video), and `-s` sets the default shell. Set a password for this new user with `passwd username`.

To allow the new user to execute administrative commands using `sudo`, you need to install the `app-admin/sudo` package and edit the `sudoers` file. Install it with `emerge app-admin/sudo`. Then, edit the `sudoers` file using the `visudo` command. This tool ensures the file is edited safely without corrupting its syntax. Within the `sudoers` file, you will find a line that looks like `%wheel ALL=(ALL) ALL`. This grants all members of the `wheel` group full `sudo` privileges. Since you added your user to the `wheel` group earlier, they will now be able to use `sudo` for administrative tasks.

With the user accounts configured, the next step is to install essential system components. If you plan to use a graphical desktop environment, you will need to install an X server, a display manager, and the desktop environment itself. For example, to install the Xorg server and the KDE Plasma desktop, you would run `emerge -av x11-base/xorg-server kde-plasma/plasma-desktop`. You would also want to enable the display manager (like `xdm` or `sddm`) to start at boot using `systemctl enable sddm.service`. For a headless server, you might install a web server or other daemons instead.

Before considering the installation complete, a thorough verification of the ZFS setup is mandatory. Reboot the system and pay close attention to the boot process. Does ZFSBootMenu appear? Can you enter the emergency shell? Is your regular boot menu visible? Successfully navigating through these steps confirms that ZBM is correctly installed and communicating with the ZFS kernel modules. Once the system boots fully to the login prompt or desktop, log in as your standard user and open a terminal. First, check that you are indeed running from the ZFS root by running `mount | grep 'on /'`. The output should show that the root filesystem is mounted from a device of type `zfs`. Next, verify the status of your ZFS pool with `zpool status`. All devices should show as online and healthy. Check that your datasets are mounted correctly by running `zfs list`. You should see `rpool`, `rpool/R00T/default`, and your other datasets listed, with the correct mountpoints.

A key test of the system's stability is to simulate a reboot. Simply run the `reboot` command. The system should shut down cleanly and power back on, passing through ZFSBootMenu again and booting into the same state. This end-to-end test validates the entire chain: from UEFI firmware calling ZBM, to ZBM importing and unlocking the encrypted pool, to the kernel mounting the ZFS root, to systemd starting the user session. Any failure during this cycle indicates a misconfiguration in one of the preceding steps, such as a missing init service, an incorrect ZBM configuration, or a problem with the `hostid`.

Finally, it is highly recommended to install a fallback initramfs generator like `dracut` as an emergency safety net. While ZFSBootMenu is robust, having a secondary boot path is prudent. Install it with `emerge sys-kernel/dracut`. Then, create a configuration file `/etc/dracut.conf.d/zfs.conf` with the content `add_dracutmodules+= " zfs "`. This tells dracut to include the necessary ZFS modules in any initramfs it generates. Finally, generate the fallback initramfs with `dracut - - force`. This initramfs can be booted from the UEFI firmware's boot menu (if configured) and should be able to import your pool and decrypt it manually, providing another avenue for recovery if ZBM were to fail. You can also add a ZFSBootMenu entry for this dracut-generated image if desired, further enhancing the system's resilience.

By meticulously completing these final steps—configuring user accounts, installing necessary software, and rigorously verifying the boot process—you transform the constructed system from a collection of files and partitions into a fully operational and reliable Gentoo Linux workstation. The combination of ZFS' data integrity, native encryption, and snapshot capabilities, managed by the intelligent ZFSBootMenu, results in a powerful and flexible computing environment tailored precisely to the user's advanced requirements.

Reference

1. Put the fun back into computing. Use Linux, BSD. - DistroWatch.com <https://distrowatch.com/?distribution=&month=January&year=all>
2. Put the fun back into computing. Use Linux, BSD. - DistroWatch.com <https://distrowatch.com/index.php?month=January&a>

3. Search Distributions - DistroWatch.com <https://distrowatch.com/search.php?ostype=Linux&category=All&origin=All&basedon=All&basedon=Slackware&desktop=All&architecture=All&package=All&rolling=All&isostage=All&defaultinit=All&status=All>
4. suse-sles-hpc-15-sp3-byos-v20210924-hvm-ssd-x86_64 Package ... https://publiccloudimagechangeinfo.suse.com/amazon/suse-sles-hpc-15-sp3-byos-v20210924-hvm-ssd-x86_64/package_changelogs.html
5. Automate Linux SysAdmin tasks with Ansible in 100+ examples <https://www.udemy.com/course/automate-linux-tasks-with-ansible/?srsltid=AfmBOopzaDDwZqGStPehFRREQuN1NiTz61n3YiOmVL6AZ0JsqcTVcBS3>
6. Search Distributions - DistroWatch.com <https://distrowatch.com/search.php?basedon=Ubuntu&status=All>
7. Search Distributions - DistroWatch.com <https://distrowatch.com/search.php?origin=Germany>
8. ZFSBootMenu not booting to kernel - Practical ZFS <https://discourse.practicalzfs.com/t/zfsbootmenu-not-booting-to-kernel/4194>
9. Using a YubiKey to unlock LUKS and Root on ZFS with native ... <https://www.alexmoch.com/blog/gentoo-yubikey-encrypted-root-on-zfs/>
10. [PDF] PDF - cloud-init - Read the Docs https://cloudinit.readthedocs.io/_/downloads/en/21.1/pdf/
11. 4.6. Booleans | SELinux User's and Administrator's Guide https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guide/sect-security-enhanced_linux-working_with_selinux-booleans
12. Chapter 3. Targeted Policy | SELinux User's and ... https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/selinux_users_and_administrators_guide/chap-security-enhanced_linux-targeted_policy
13. Really weird issue when booting from zfsbootmenu <https://discourse.practicalzfs.com/t/really-weird-issue-when-booting-from-zfsbootmenu/2559>
14. Multiboot Raspberry Pi with PINN OS | PDF | Linux - Scribd <https://www.scribd.com/document/977081193/Linux-Magazine-USA-Issue-302-January-2026>
15. 533 reader reviews of openSUSE... - DistroWatch.com <https://distrowatch.com/dwres.php?resource=ratings&distro=opensuse>
16. [PDF] Release 1.1.5 The Borg Collective https://borgbackup.readthedocs.io/_/downloads/en/1.1.5/pdf/
17. Need help with zfsbootmenu - OpenZFS <https://discourse.practicalzfs.com/t/need-help-with-zfsbootmenu/4315>

18. LINUX Unplugged - TopPodcast.com https://toppodcast.com/podcast_feeds/linux-unplugged/
19. ChangeLog-SLE-15-SP4-GM-SLE- ... <https://www.suse.com/c/wp-content/uploads/2023/04/ChangeLog-SLE-15-SP4-GM-SLE-15-SP5-PublicRC-202304.txt>
20. Understanding SELinux Basics | SUSE Linux Micro 6.1 <https://documentation.suse.com/sle-micro/6.1/html/Micro-selinux/selinux-article.html>
21. profiles/use.local.desc - NJU Git <https://git.nju.edu.cn/ti/gentoo/-/blob/47081c622bfb4c4aa730652f1fb0c101cfc51f91/profiles/use.local.desc>
22. Understanding nvidia-drm.modeset=1 (NVIDIA Linux driver ... <https://forums.developer.nvidia.com/t/understanding-nvidia-drm-modeset-1-nvidia-linux-driver-modesetting/204068>
23. Installation NVIDIA drivers on Gentoo failed - Linux <https://forums.developer.nvidia.com/t/installation-nvidia-drivers-on-gentoo-failed/283965>
24. (CachyOS kernel 6.18.2-3-cachyos) Extra monitors not detected ... <https://forums.developer.nvidia.com/t/cachyos-kernel-6-18-2-3-cachyos-extra-monitors-not-detected-nvidia-smi-returns-no-devices-were-found/355217>
25. FYI FWIW - Building zfsbootmenu with remote SSH access ... <https://discourse.practicalzfs.com/t/fyi-fwiw-building-zfsbootmenu-with-remote-ssh-access-and-zfs-version-2-2-6/1850>